

Rtl2lean: Automated RTL-to-Lean Translation with Hierarchical Theorem Generation and Lemma Reuse

Hongqin Lyu^{1,2}, Junxing Dong³, Yonghao Wang¹, Zhiteng Chao¹, Tiancheng Wang^{1,2} and Huawei Li^{1,2}

¹State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing, China

²University of Chinese Academy of Sciences, Beijing, China

³NorthChina University of Science and Technology

Abstract—Formal verification with interactive theorem provers can provide strong correctness guarantees for register transfer level designs, but applying it to existing SystemVerilog code requires substantial manual effort in semantic modeling and proof construction. This paper presents Rtl2lean, a framework that automatically translates RTL designs into executable Lean 4 models and builds a hierarchical theorem library for subsequent verification. The generated model represents hardware execution as a pure state transition function, while a four layer theorem framework captures combinational semantics, sequential updates, single cycle behavior, and reachability and invariants. When a high level property cannot be discharged by the existing theorem base, an LLM based proving loop proposes intermediate lemmas from the current proof context and Lean feedback. Only lemmas accepted by the Lean kernel are added to the reusable lemma pool. Experiments on six SystemVerilog designs generate 403 theorems, all of which are successfully checked by Lean. Among 358 foundational lemmas, 287 are available for automatic reuse, yielding a reusable lemma ratio of 80.2 percent. The results demonstrate that Rtl2lean can construct machine checked RTL proof libraries with low checking overhead and substantial cross property lemma reuse.

Index Terms—RTL verification, Lean 4, theorem proving, lemma generation, large language models

I. INTRODUCTION

As integrated circuits continue to grow in scale and complexity, register-transfer level (RTL) verification has become one of the most critical and costly stages in modern hardware development [1]. Processors and system-on-chip designs often contain complex datapaths, control state machines, pipelined structures, and a large number of parameterized modules. Even a minor error, such as a missing state transition, an unhandled boundary condition, or an incorrect bit-vector operation, may remain undetected until system integration or even after tape-out, leading to substantial debugging and repair costs. Therefore, establishing rigorous and scalable correctness guarantees for RTL designs at an early stage is a central challenge in hardware design automation.

Existing RTL verification flows mainly rely on dynamic simulation, model checking, equivalence checking, and formal verification based on SAT/SMT solvers. Simulation is practical and scalable for engineering use, but it can examine only a finite set of input vectors and execution traces. Model checking and bounded model checking can automatically explore the state space and have been widely used for both hardware and software verification [2], [3]. However, as the number of

registers, pipeline depth, and overall state space increase, the solving process is often limited by state explosion and the rapid growth of constraints [4].

Unlike automated model checking, which relies on exploring a finite state space, interactive theorem provers such as Lean 4 and Coq allow users to define hardware semantics in formal logic and construct machine-checked proofs that hold for arbitrary inputs and execution lengths. Lean 4 further provides dependent types, extensible metaprogramming, and efficient code generation, making it suitable both for expressing RTL designs and their properties and for building domain-specific proof automation [5]. Prior work such as Kami and Silver Oak has shown that embedding hardware semantics in proof assistants can support modular and parameterized verification while providing stronger correctness guarantees than finite simulation and bounded checking [6], [7]. However, applying this approach to existing RTL designs remains costly. The event-driven semantics of the verilog differ substantially from Lean’s reasoning model [8] based on pure functions and type theory, while proofs of complex hardware properties often require manually constructed auxiliary definitions, invariants, and intermediate lemmas.

Fortunately, large language models (LLMs) have recently demonstrated strong heuristic reasoning capabilities in both code generation and formal theorem proving. Early work such as GPT-f showed that generative language models can be used to predict formal proof steps [9]. LeanDojo further introduced infrastructure for interacting with the Lean proof environment, extracting proof data, and retrieving relevant premises, and demonstrated that retrieval-based premise selection can improve automated theorem proving [10]. In the hardware domain, the recent CktFormalizer [11] framework translates natural-language hardware specifications into type-safe hardware descriptions embedded in Lean 4, and uses Lean for compilation checking and functional equivalence proofs. Together, these studies suggest that LLMs can identify promising reasoning steps from the current proof state and existing proof libraries, thereby assisting theorem provers in proof search.

However, this paper focuses on a different yet complementary question: **Can existing RTL designs be automatically translated into a Lean 4 formal library, while large language models dynamically generate intermediate lemmas to improve the automation of RTL theorem proving?**

Achieving this goal involves two main challenges. First, there is currently no tool that can automatically translate RTL into Lean 4 representations. Because RTL behavior is jointly determined by current inputs, register states, and combinational and sequential logic, the resulting Lean model must hide unnecessary Verilog execution details while faithfully preserving the cycle-level semantics of the original design. Second, proofs of complex RTL properties often rely on highly design-specific intermediate lemmas, such as state invariants, bit-vector properties, and multi-cycle relations. These lemmas are difficult to enumerate in advance and are also challenging for existing proof automation techniques to generate directly.

To address these challenges, we propose *Rtl2lean*, an LLM-assisted Lean 4 formal verification framework for RTL designs. First, we develop a lightweight RTL-to-Lean 4 compiler that translates combinational logic, state updates, and bit-vector operations into Lean definitions with explicit cycle-level semantics, providing a unified representation for formally specifying and proving RTL properties. Second, we integrate an LLM into Lean’s interactive proof loop. When a proof attempt fails, the framework collects the current proof goal, relevant definitions, and error feedback, and uses the LLM to generate candidate intermediate lemmas. Each generated lemma must be verified by the Lean kernel before it can be used to advance the proof.

The main contributions of this work are as follows:

- 1) An automated RTL-to-Lean translation flow is proposed to convert combinational logic, sequential state updates, and fixed-width bit-vector operations into Lean definitions with explicit cycle-level semantics.
- 2) An LLM-assisted theorem proving mechanism is introduced to generate intermediate lemmas from the current proof goal, local context, and Lean error feedback through an iterative generate–check–repair process.
- 3) Experiments on six SystemVerilog designs demonstrate a 100 percent proof success rate and an 80.2 percent reusable lemma ratio.

II. BACKGROUND

A. Interactive Theorem Proving

The goal of theorem proving is to establish that a proposition follows from a set of formally specified assumptions through valid logical inference. Let Γ denote a logical context containing variables, assumptions, definitions, and previously proved results, and let P denote the proposition to be proved. A proof task can be written as

$$\Gamma \vdash P, \quad (1)$$

where $\vdash$ indicates that P can be derived from the information contained in Γ using valid inference rules. A formal proof is therefore a sequence of logically valid steps that transforms the initial goal into propositions that can be directly established from the context.

Lean 4 is an interactive theorem prover based on dependent type theory and the propositions-as-types principle. Under

this principle, a proposition is represented as a type, while a proof of that proposition is represented as a term of the corresponding type. Proving a proposition P is therefore equivalent to constructing a proof term p such that

$$\Gamma \vdash p : P. \quad (2)$$

The Lean kernel checks whether the constructed term indeed has type P . A theorem is accepted only if its proof term passes kernel type checking.

B. Definitions, Theorems, and Intermediate Lemmas

A formal development is primarily composed of definitions, theorems, and lemmas. Definitions specify the objects being reasoned about and their formal semantics. Theorems express the final correctness properties to be established, while lemmas capture intermediate results that can be reused during the proof.

Let T be the target theorem. When directly proving T is difficult, the proof can be decomposed by first establishing a set of intermediate lemmas,

$$L_1, L_2, \dots, L_n, \quad (3)$$

and then using them to derive the final result:

$$\Gamma \vdash L_1, \quad \Gamma \vdash L_2, \quad \dots, \quad \Gamma \vdash L_n, \quad (4)$$

$$\Gamma, L_1, L_2, \dots, L_n \vdash T. \quad (5)$$

Intermediate lemmas do not alter the meaning of the target theorem. Instead, they expose auxiliary properties required by the proof and decompose a complex reasoning task into smaller logical obligations.

III. FRAMEWORK OF RTL2LEAN

Before presenting the technical details, we first introduce the overall workflow of *rtl2lean*, as illustrated in Fig. 1. The framework consists of three stages.

- 1) *RTL-to-Lean semantic compilation*: translates SystemVerilog RTL into a pure functional state-transition model in Lean 4
- 2) *Theorem framework generation*: constructs a four-layer proof skeleton covering combinational logic, sequential processes, single-cycle behavior, and reachability or invariants.
- 3) *Autonomous proving with lemma reuse*: proves high-level specifications through a Lean-guided feedback loop and stores verified theorems in a lemma pool for subsequent proof tasks.

A. RTL-to-Lean Semantic Compilation

SystemVerilog combines width-sensitive operations, blocking and non-blocking assignments, conditional control flow, and distinct combinational and sequential processes, making a direct encoding into a theorem prover difficult to reason about. To address this issue, *rtl2lean* first parses the RTL design with PySlang and constructs a typed abstract syntax tree. The resulting syntax tree is then lowered into an intermediate

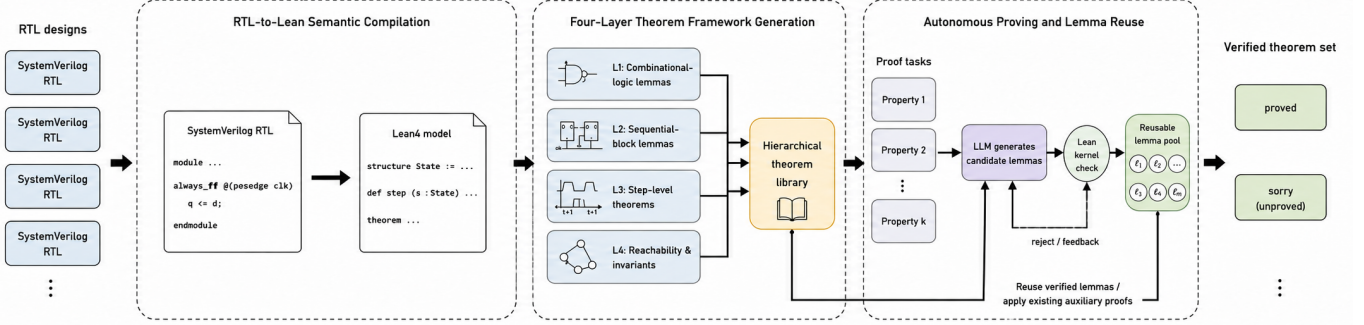


Fig. 1. RTL2LEAN framework.

representation, where conditional updates are normalized as selection expressions and different assignment types retain their corresponding update semantics. After standard optimizations, such as constant folding, dead-code elimination, and common-subexpression elimination, the backend generates a Lean 4 model consisting of State, Inputs, Outputs, and a single-cycle transition function `step`.

The generated model represents hardware execution as a pure function. Given the current state s and input i , $\text{step}(s, i)$ returns the next state without side effects. For example, the behavior of a counter with reset and enable signals can be abstracted as

$$\text{step}(s, i) = \begin{cases} \text{init}, & i.\text{rst} = \text{true}, \\ s[\text{count} \leftarrow s.\text{count} + 1], & i.\text{enable} = \text{true}, \\ s, & \text{otherwise.} \end{cases} \quad (6)$$

This functional encoding preserves the cycle-level transition semantics of the RTL design while exposing an explicit state relation on which subsequent theorems can be constructed.

B. Four-Layer Theorem Framework Generation

Although RTL-to-Lean compilation establishes an executable formal semantics for the design, directly proving high-level specifications may still require repeated unfolding of combinational functions, sequential processes, and the single-cycle transition function. Such proofs quickly become verbose and contain substantial redundant reasoning. To address this issue, *rtl2lean* automatically constructs a bottom-up four-layer theorem framework from the generated Lean 4 model. Lower layers capture local hardware semantics, while higher layers reuse these results to establish cycle-level and multi-cycle properties.

1) *Combinational-Logic Lemmas*: The first layer targets functions generated from continuous assignments and combinational processes. For each combinational function, the framework generates semantic lemmas for updated fields and preservation lemmas for unaffected fields. As illustrated in

Fig. 2(a), the semantic lemma characterizes how the overflow flag is computed, while the preservation lemma states that the counter register remains unchanged. The preservation lemma is marked with Lean’s `@[simp]` attribute, allowing the simplifier to apply it automatically in subsequent proofs.

2) *Sequential-Block Lemmas*: The second layer targets state-update functions generated from sequential processes such as `always_ff`. For each sequential function, the framework derives determinism lemmas, preservation lemmas for registers outside its write set, and reset-behavior lemmas when a reset pattern is identified. Figure 2(b) shows a counter update, its functional determinism property, and preservation of an unwritten overflow register. These lemmas package register-level behavior into reusable proof units and avoid repeatedly unfolding the complete sequential function.

3) *Step-Level Theorems*: The third layer is built around the unified single-cycle transition function `step`, which combines reset handling, combinational evaluation, and sequential state updates. This layer captures module-level properties such as reset behavior, transition determinism, output reflection, and finite-state machine transitions. In Fig. 2(c), reset assertion forces the next state to equal the initial state, while an output-reflection lemma relates the externally observed count value to the corresponding internal register. Step-level theorems therefore provide a direct interface for reasoning about the complete behavior of one hardware cycle.

4) *Reachability and Invariants*: The first three layers characterize individual functions or a single transition step, whereas many hardware specifications concern executions of arbitrary length. The fourth layer therefore introduces reachability to lift the single-cycle semantics of `step` to multi-cycle executions. As shown in Fig. 2(d), a state is reachable if it can be obtained from the initial state by folding `step` over a finite input sequence. The framework then proves that reachability is preserved by one additional transition and introduces state invariants over all reachable states. Such results provide an induction basis for proving higher-level safety and functional specifications.

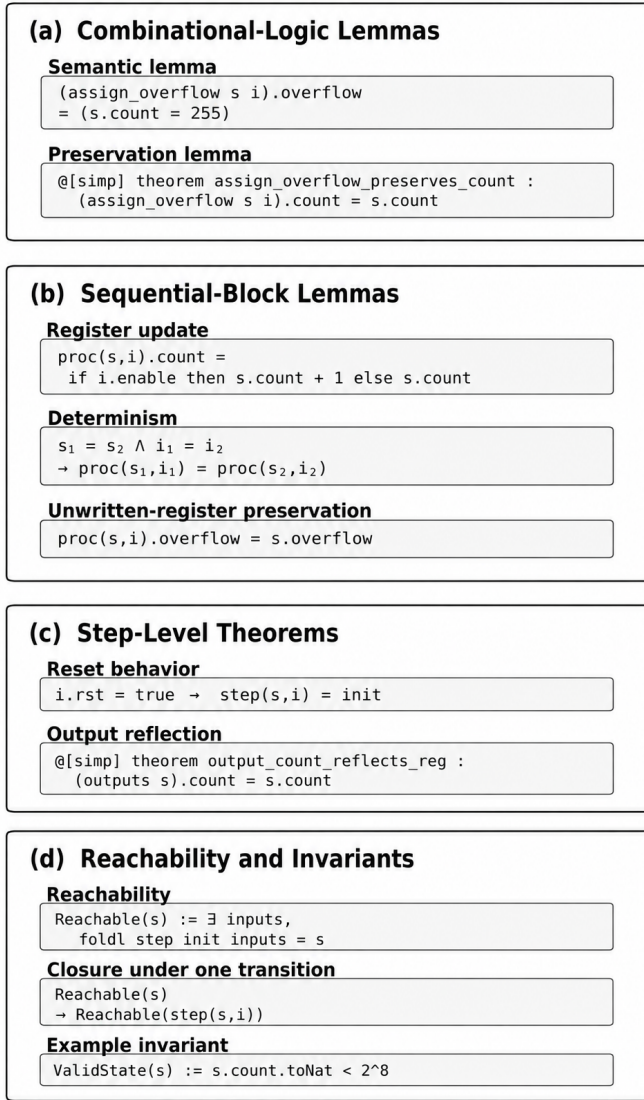


Fig. 2. Representative examples generated by the four-layer.

C. Autonomous Proving and Lemma Reuse

The previous stages generate an executable Lean 4 model and a layered theorem base. However, high-level functional specifications often require design-specific intermediate results that cannot be derived by fixed proof templates alone. To address this issue, *Rtl2lean* employs a Lean-guided autonomous proving procedure. When the existing theorem base cannot discharge the current goal, an LLM generates candidate intermediate lemmas from the proof context and Lean error feedback. Only candidates accepted by the Lean kernel are added to the lemma pool and reused in subsequent proof tasks.

1) *Proof-Task Construction*: For each high-level specification, the system first creates a Lean theorem with an incomplete proof and collects the relevant context, including the generated state definitions, transition functions, four-layer theorem base, and verified lemmas stored in the lemma pool.

For a target theorem T , the proof context is represented as

$$\mathcal{C}(T) = (M, T, \mathcal{L}_{\text{base}}, \mathcal{L}_{\text{pool}}), \quad (7)$$

where M denotes the generated Lean 4 model, $\mathcal{L}_{\text{base}}$ is the set of automatically generated lower-layer lemmas, and $\mathcal{L}_{\text{pool}}$ contains reusable lemmas proved in earlier tasks.

For example, a high-level property for the counter states that the register remains unchanged when neither reset nor enable is asserted:

$$i.\text{rst} = \text{false} \wedge i.\text{enable} = \text{false} \implies \text{step}(s, i).\text{count} = s.\text{count}. \quad (8)$$

The system provides this target together with the definition of `step`, the sequential-update semantics, and relevant preservation lemmas.

2) *Lean-Guided Proving and Lemma Reuse*: Given the constructed proof task, the system first attempts to solve the target using the existing theorem base. If the proof remains incomplete, an LLM proposes candidate intermediate lemmas according to the current goal, relevant definitions, available lemmas, and Lean error messages. Each candidate is checked by the Lean kernel before use. Rejected candidates are returned to the LLM with diagnostic feedback, whereas accepted lemmas are added to the lemma pool and used to continue the proof. The iterative process is expressed as

$$p_{k+1} = \text{Generate}(\mathcal{C}(T), e_k), \quad (9)$$

where p_k is the k -th candidate proof and e_k is the error returned by Lean. The loop terminates when

$$\text{Check}(T, p_k) = \text{Pass}. \quad (10)$$

After a successful proof, the lemma pool is updated as

$$\mathcal{L}_{k+1} = \mathcal{L}_k \cup \{T\} \cup \mathcal{H}, \quad (11)$$

where $\mathcal{H}$ denotes auxiliary lemmas or local `have` statements discovered during proof generation. Later proofs can reuse these results through tactics such as `rw`, `apply`, and `simp`, avoiding repeated unfolding of the underlying RTL semantics.

For the counter property in Eq. (8), a valid proof can be written as

```
theorem count_stable_when_disabled :
  forall (s : counterState) (i : counterInputs),
    i.rst = false ->
    i.enable = false ->
    (step s i).count = s.count := by
  intro s i hrst hen
  simp [step, proc_alwaysff, hrst, hen]
```

Once verified, this theorem can be reused to prove related properties, such as the stability of the corresponding output value.

IV. EXPERIMENTAL

A. Experimental set

To evaluate the effectiveness of *Rtl2lean* in RTL formal modeling, theorem generation, and Lean-based machine checking, we select six SystemVerilog designs with diverse functional and structural characteristics from OpenCores [12],

TABLE I
OPENCORES RTL DESIGNS USED IN THE EXPERIMENTS

Design	Main Function
debounce_edge	Debouncing and edge detection
fir_filter	Finite impulse response filtering
pwm_generator	Pulse-width modulation
lfsr_crc	LFSR and CRC computation
sync_fifo	Synchronous FIFO buffering
ps2_keyboard	PS/2 keyboard control

as shown in Table I. This benchmark suite enables the applicability of Rtl2lean to be evaluated across different design types and hardware structures. To validate the effectiveness of the proposed method, Lean 4.26 [13] is used to check the Lean models, foundational lemmas, and high-level properties generated by Rtl2lean. All experiments are conducted on a server equipped with an Intel(R) Xeon(R) Gold 6148 CPU and 629 GB of RAM.

B. Evaluation Metrics

To evaluate the proof-generation capability, correctness, execution efficiency, and lemma reusability of Rtl2lean, we adopt the metrics summarized in Table II. The total number of theorems, denoted by N_{total} , measures the overall scale of the formal verification artifacts generated and checked by Lean. The number of high-level properties, denoted by N_{property} , indicates the number of RTL functional and state-related properties verified by the framework. The number of foundational lemmas, denoted by N_{lemma} , characterizes the size of the supporting proof library automatically generated for proving these high-level properties.

The proof success rate measures the proportion of generated theorems that are successfully accepted by Lean and is defined as

$$\text{Proof Success Rate} = \frac{N_{\text{proved}}}{N_{\text{total}}} \times 100\%, \quad (12)$$

where N_{proved} denotes the number of theorems successfully checked by Lean, and N_{total} denotes the total number of generated theorems. We additionally report the numbers of errors, unresolved goals, and proofs containing `sorry` to identify incomplete verification results.

For efficiency evaluation, we report the end-to-end execution time, denoted by T_{e2e} , and the Lean checking time, denoted by T_{Lean} . The former measures the total time for RTL translation, theorem and lemma generation, proof construction, and result collection, while the latter measures the time required by Lean to check the generated proof files. We also evaluate the automatic reusability of foundational lemmas by defining the reusable lemma ratio as

$$R_{\text{reuse}} = \frac{N_{\text{reusable}}}{N_{\text{lemma}}} \times 100\%, \quad (13)$$

where N_{reusable} denotes the number of foundational lemmas that can be automatically applied in subsequent proofs, and N_{lemma} denotes the total number of foundational lemmas.

TABLE II
EVALUATION METRICS FOR RTL2LEAN

Metric	Symbol
Total theorems	N_{total}
High-level properties	N_{property}
Foundational lemmas	N_{lemma}
Proof success rate	R_{proof}
End-to-end time	T_{e2e}
Lean checking time	T_{Lean}
Reusable lemma ratio	R_{reuse}

C. Proof Results and Lemma Reuse Analysis

Table III presents the proof results and lemma reuse statistics of Rtl2lean on the six RTL designs. Rtl2lean generates and verifies a total of 403 theorems, including 45 high-level properties and 358 foundational lemmas. Among the foundational lemmas, 287 can be automatically reused in subsequent proofs. All generated theorems are successfully checked by Lean, resulting in a proof success rate of 100% for every design.

As shown in Table III, the number of high-level properties remains relatively stable across the evaluated designs, ranging from seven to eight, whereas the number of foundational lemmas varies considerably, from 22 for `fir_filter` to 86 for `sync_fifo`. This difference indicates that the size of the foundational proof library is mainly affected by the complexity of state variables, assignment operations, and state-update dependencies, rather than solely by the number of high-level properties. In particular, `sync_fifo` contains more state and control relations and therefore produces the largest number of foundational lemmas and total theorems.

All six designs achieve a proof success rate of 100%, demonstrating that the Lean models and proofs generated by Rtl2lean can be successfully machine-checked across designs with different functions and structures. The overall reusable lemma ratio reaches 80.2%, with five designs exceeding 77%, indicating that most foundational lemmas can be reused in subsequent high-level proofs to reduce repeated reasoning over low-level assignment and state-update semantics. This result also validates the effectiveness of Rtl2lean’s hierarchical proof structure, in which foundational semantic relations are proved once and then shared across multiple properties. Note that R_{reuse} measures the proportion of lemmas available for automatic reuse rather than their exact dynamic usage in individual proofs.

D. Time Overhead

We measure the end-to-end execution time, T_{e2e} , and the Lean checking time, T_{Lean} . The former includes RTL translation, theorem and lemma generation, proof construction, and result collection, while the latter measures the time required by Lean to check the generated proof files. Table IV reports the checking time for each design.

The complete proof-generation pipeline takes only $T_{\text{e2e}} = 0.650$ s for all six designs, since the deterministic proof

TABLE III
PROOF RESULTS AND LEMMA REUSE STATISTICS OF RTL2LEAN

Design	N_{property}	N_{lemma}	N_{reusable}	N_{total}	R_{proof}	R_{reuse}
debounce_edge	8	71	56	79	100%	78.9%
fir_filter	7	22	14	29	100%	63.6%
pwm_generator	7	74	62	81	100%	83.8%
lfsr_crc	7	35	27	42	100%	77.1%
sync_fifo	8	86	70	94	100%	81.4%
ps2_keyboard	8	70	58	78	100%	82.9%
Total	45	358	287	403	100%	80.2%

TABLE IV
TIME OVERHEAD OF RTL2LEAN

Design	N_{total}	T_{Lean} (s)
debounce_edge	79	1.113
fir_filter	29	0.558
pwm_generator	81	0.918
lfsr_crc	42	0.721
sync_fifo	94	0.936
ps2_keyboard	78	1.327
Total	403	5.573

templates require neither proof search nor external model calls. In comparison, Lean checking takes 5.573 s in total, indicating that most of the time is spent on definition reduction and proof-term checking. Each design is checked in less than 1.4 s, including `sync_fifo`, which contains the largest number of theorems. However, T_{Lean} is not strictly proportional to N_{total} ; for example, `ps2_keyboard` requires more checking time than `sync_fifo` despite containing fewer theorems. This suggests that checking cost is also influenced by the complexity of state-transition logic and bit-vector expressions.

V. CONCLUSION

This paper presents `Rtl2lean`, a framework that translates SystemVerilog RTL into executable Lean 4 models, generates a hierarchical theorem library, and supports high-level verification through LLM-assisted lemma generation and Lean-checked lemma reuse. Experiments on six RTL designs produce 403 machine-checked theorems with a 100 percent proof success rate and an 80.2 percent reusable lemma ratio. These results demonstrate the potential of combining automated RTL translation, structured theorem generation, and kernel-checked LLM assistance for scalable hardware verification.

REFERENCES

- [1] H. D. Foster, “Trends in functional verification: A 2014 industry study,” in *Proceedings of the 52nd Annual Design Automation Conference*, ser. DAC ’15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 48:1–48:6.
- [2] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Cham: Springer, 2018.

- [3] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu, “Symbolic model checking without BDDs,” in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, W. R. Cleaveland, Ed., vol. 1579. Berlin, Heidelberg: Springer, 1999, pp. 193–207.
- [4] E. M. Clarke, W. Klieber, M. Nováček, and P. Zuliani, “Model checking and the state explosion problem,” in *Tools for Practical Software Verification*, ser. Lecture Notes in Computer Science, B. Meyer and M. Nordio, Eds. Berlin, Heidelberg: Springer, 2012, vol. 7682, pp. 1–30.
- [5] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Automated Deduction – CADE 28*, ser. Lecture Notes in Computer Science, A. Platzer and G. Sutcliffe, Eds., vol. 12699. Cham: Springer, 2021, pp. 625–635.
- [6] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind, “Kami: A platform for high-level parametric hardware specification and its modular verification,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. ICFP, pp. 24:1–24:30, 2017.
- [7] Project Oak, “Silver Oak: Formal specification and verification of hardware,” <https://project-oak.github.io/silveroak/>, accessed: 2026-07-17.
- [8] IEEE, “IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language,” Institute of Electrical and Electronics Engineers, New York, NY, USA, IEEE Standard IEEE Std 1800-2023, Feb. 2024, revision of IEEE Std 1800-2017.
- [9] S. Polu and I. Sutskever, “Generative language modeling for automated theorem proving,” *arXiv preprint arXiv:2009.03393*, 2020.
- [10] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar, “LeanDojo: Theorem proving with retrieval-augmented language models,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 21 573–21 612.
- [11] J. Xiong, Q. Han, C. Ding, H. Xiao, Z. Su, C. Tao, and N. Wong, “CktFormalizer: Autoformalization of natural language into circuit representations,” *arXiv preprint arXiv:2605.07782*, 2026.
- [12] OpenCores, “OpenCores: Free and open-source gateway IP cores,” <https://opencores.org/>, online resource, accessed July 18, 2026.
- [13] Lean FRO, “Lean 4.26.0 release notes,” <https://lean-lang.org/doc/reference/latest/releases/v4.26.0/>, Dec. 2025, released December 13, 2025; accessed July 18, 2026.